Revit Accessibility Pre-Check

Complete project guide: plan, architect workflow, developer implementation, rule verification, and test cases.

Status: planning complete. No regulation threshold is verified yet. The tool is a pre-check and does not certify compliance.

Revit Accessibility Pre-Check (Sample Project)

A pre-check tool that reads a Revit model and reports elements that may not meet accessibility requirements, with the regulation clause cited for each finding.

This tool is a pre-check. It does not certify compliance. Every result must be reviewed by a qualified professional before it is relied on.

Status

Item	Status
Project plan and guides	Complete (this folder)
Rule register (thresholds)	**Placeholders only.** No value is verified yet.
Architect workflow (Dynamo + Excel)	Documented; graph not yet built
Core rule engine (C#)	Implemented with door clear-width rule; 20 tests pass (see `src/`)
Revit add-in (C#)	Planned; not yet implemented

Who this is for

Reader	Start here
Architect or compliance reviewer (no coding)	`docs/02-architect-guide-dynamo.md`
Developer building the add-in	`docs/03-developer-guide-csharp.md`
Anyone verifying rule values	`docs/04-rule-verification-checklist.md`
Project owner planning next steps	`docs/05-roadmap-and-decisions.md`
Overview of the whole project	`docs/01-project-overview.md`

Folder contents

```
revit-accessibility-precheck/
  README.md                          this file
  docs/
    01-project-overview.md           what the tool does, architecture, risks
    02-architect-guide-dynamo.md     step-by-step for non-programmers (Dynamo)
    03-developer-guide-csharp.md     step-by-step for the C# add-in
    04-rule-verification-checklist.md how to verify each regulation value
    05-roadmap-and-decisions.md      phases, decisions, open questions
  rules/
    rule-register.csv                the rule table (edit values here, not in code)
  test-model/
    known-results-test-cases.md      reference cases to validate the checker
```

Before publishing to GitHub

- Do not commit client or friend project files (`.rvt`, `.rfa`, `.ifc`).
- Do not commit Autodesk DLLs. Use the Revit API NuGet package instead.
- Keep every unverified threshold marked as a placeholder.
- Choose a licence (MIT is a common choice for a sample).

Licence

Not yet chosen. Add a `LICENSE` file before publishing. Autodesk Revit API use is governed by Autodesk's own terms.

1. Project Overview

1.1 Purpose

Architects and designers need to find accessibility problems early, while a model is still cheap to change. This project provides a pre-check that:

1. Reads building elements from a Revit model (starting with doors).
2. Compares measured values with the thresholds in a rule register.
3. Reports each element as **Pass**, **Fail**, or **Cannot evaluate**.
4. Cites the regulation clause behind every finding.
5. Lets the user see, filter, and export the results.

1.2 What it is not

- It is not a certificate of compliance.
- It is not a replacement for a professional review.
- It does not change the model.
- It does not check rules that need information the model does not contain.

1.3 Three result types

Status	Meaning	Action
Fail	The measured value breaks a rule threshold.	Review and fix, or record why not.
Cannot evaluate	The model lacks the data needed to decide.	Add the parameter or fix the family.
Pass	The value meets the threshold.	None (hidden by default).

"Cannot evaluate" must never be shown as "Pass". A silent pass is the most dangerous failure of a compliance tool.

1.4 Delivery options

Option	Who builds it	Who maintains it	Use when
A. Dynamo graph + Excel rule register	Developer builds once	Architects edit Excel	Immediate use by design teams
B. C# Revit add-in with a tested rule engine	Developer	Developer	Long-term product, shipped to others
C. pyRevit Python script	Developer	Developer	Quick scripting alternative to C#
D. Commercial checker (e.g., Solibri)	Vendor setup	Vendor and firm	Firm already owns the software

The recommended sequence is **A first**, because it needs no software development for day-to-day use, then **B** once the rules are stable.

1.5 Architecture (for option B)

```
Revit model
   |
   v
Adapter (Revit API)      reads doors, parameters, levels; converts units
   |
```

```
   v
Neutral model (DTOs)     plain data, no Revit references
   |
   v
Rule engine (Core)       applies IRule implementations; returns Findings
   |
   v
Results UI               grid, filters, detail pane, CSV export
```

Key rule: the **Core** library must not reference any Autodesk assembly. This is what makes the rules testable without Revit installed.

1.6 Main risks

Risk	Effect	Mitigation
Wrong threshold value	Wrong results for every project	Verification checklist (doc 04); no value used unverified
Inconsistent family parameters	Many "Cannot evaluate" results	Model-quality pre-check; parameter mapping per firm
Designers rely on the tool as certification	Liability and safety issues	Permanent disclaimer; "pre-check" wording
Revit version differences	Add-in or Dynamo graph breaks	Lock Revit version per project
Client data published on GitHub	Confidentiality breach	Strict .gitignore; sample models only
Tool ignored because too noisy	Problems not caught	Clear severity; hide passes; stable rule set

1.7 Glossary

- **BIM**: Building Information Modeling.
- **Clear width**: the usable opening width of a door, after the frame.
- **Dynamo**: visual programming tool included with Revit.
- **DTO**: a plain data object passed between layers.
- **Rule register**: the table of thresholds, clauses, and severities.
- **BCF**: BIM Collaboration Format, an open standard for exchanging issues.

2. Architect Guide: Running the Check with Dynamo (No Coding)

This guide is for architects and compliance reviewers. You do not need to write any code.

2.1 What you need

- Revit 2026 with the project open.
- Dynamo (included with Revit; open it from the Manage tab, or the Dynamo icon).
- Dynamo Player (included with Dynamo; runs a finished graph without editing it).
- The rule register file `rules/rule-register.csv`, opened in Excel.
- The Dynamo graph file (`.dyn`), provided by the developer. *The graph is not built yet; see section 2.7.*

2.2 Step 1: Quick check with a Revit schedule (10 minutes, no Dynamo)

Use this first to confirm the door parameter exists and to see roughly how many doors fail.

1. In Revit, go to **View > Schedules > Schedule/Quantities**.
2. Choose **Category: Doors**, then OK.
3. Add the field **Clear Width** (or the name your families use). Add **Level** and **Mark**.
4. Open the schedule's **Filter** settings and add: Clear Width is less than [threshold].

Use the threshold in the rule register (once it is verified).

1. Read the list. Each row is a door that may fail.

Limits: this shows only doors, only one rule, and no regulation citation. Use it as a first look, not as a report.

2.3 Step 2: Open the rule register

Open `rules/rule-register.csv` in Excel. Each row is one rule:

Column	Meaning
RuleID	Unique code for the rule
Clause	The regulation article it comes from
Category	The element type (e.g., Doors)
Parameter	The Revit parameter name to read
Operator	The comparison (>= or <=)
Threshold	The limit value
Unit	Usually mm
Severity	Error (Fail) or Info
Status	Placeholder, Verified, or Needs review
VerifiedBy and Date	Who checked the value, and when

Important: rows marked **Placeholder** must not be used for a real project. Their thresholds are not yet verified.

2.4 Step 3: Run the full check

1. Open the Revit project.
2. Open **Dynamo Player** (Manage tab > Visual Programming > Dynamo Player).
3. Click **Open**, choose the graph file provided by the developer, then click **Run**.

4 When the graph finishes, it will:

- Write an Excel report (default location shown in the graph).
- Colour failing elements in the active view, if the graph includes that step.

1 Note the time you ran it and the Revit file name. Keep this with the report.

2.5 Step 4: Read the report

The Excel report has one row per checked element:

Column	Meaning
ElementId	Revit element ID; use this to find the element
Level	Building level
Mark	The door tag (e.g., D-0142)
RuleID	Which rule was checked
Clause	Regulation article
Measured	The value read from the model
Required	The threshold
Status	Fail, Cannot evaluate, or Pass

How to work through it:

1 Filter **Status** to **Fail** first.

2 Sort by **Level**, then go through the doors on each floor.

3 For each Fail, find the door (by Mark or ElementId) and decide: fix it, or record why it is acceptable and who approved it.

1 Then review **Cannot evaluate** rows. These are usually a missing parameter in the door family, not a design fault. Pass them to the BIM team.

2.6 Step 5: Record your decisions

Keep a simple log next to the report (a separate sheet is best). For each Fail, record:

- Date, reviewer name, element Mark
- Decision: Fixed / Accepted with reason / Referred to specialist
- If accepted: the reason and the professional who approved it

2.7 Things you cannot do yourself (ask the developer)

- Build or change the Dynamo graph itself.
- Add a new rule type that the graph does not support.
- Fix a Dynamo error after a Revit upgrade.
- Map a new parameter name when a firm uses different family conventions.

You can change thresholds yourself, but only in the rule register, and only after the checklist in doc 04 is complete.

2.8 Troubleshooting

Problem	Likely cause	What to do
Many doors show "Cannot evaluate"	Door family lacks the Clear Width parameter	Ask BIM team to add it; check parameter name

Problem	Likely cause	What to do
Graph fails to open	Different Revit or Dynamo version	Use the Revit version the graph was built for
Report is empty	No doors in the project, or wrong category	Check the project has doors and the graph is set to Doors
Measured values look wrong (e.g., 262 instead of 800)	Unit conversion problem	Stop and report to developer; do not use the results
Results differ from schedule	Schedule and graph read different parameters	Compare the parameter names; report to developer

2.9 Golden rules for reviewers

1. The tool points to possible problems; it does not decide compliance.
2. Never use a threshold marked Placeholder.
3. "Cannot evaluate" is a data problem to fix, not a pass.
4. Keep records of every decision.

3. Developer Guide: C# Revit Add-in

Target: Revit 2026, .NET 8, Windows, developed in VS Code (CLI build) or Visual Studio.

3.1 Solution layout

```
RevitAccessibility.sln
  RevitAccessibility.Core/          class library, net8.0, NO Autodesk references
  RevitAccessibility.Core.Tests/    xUnit, net8.0
  RevitAccessibility.Revit/         class library, net8.0-windows, Revit API
  scripts/deploy.ps1                copies build output to the Revit add-in folder
```

Core must never reference a Revit assembly. If it does, its tests need Revit installed.

3.2 Create the solution

```
dotnet new sln -n RevitAccessibility
dotnet new classlib -n RevitAccessibility.Core -f net8.0
dotnet new xunit -n RevitAccessibility.Core.Tests -f net8.0
dotnet new classlib -n RevitAccessibility.Revit -f net8.0-windows
dotnet sln add RevitAccessibility.Core/RevitAccessibility.Core.csproj
dotnet sln add RevitAccessibility.Core.Tests/RevitAccessibility.Core.Tests.csproj
dotnet sln add RevitAccessibility.Revit/RevitAccessibility.Revit.csproj
dotnet add RevitAccessibility.Core.Tests reference RevitAccessibility.Core
dotnet add RevitAccessibility.Revit reference RevitAccessibility.Core
dotnet test
```

`dotnet test` should pass the template placeholder test before any code is written.

3.3 Revit API reference

Use the Revit API NuGet package that targets Revit 2026 (verify the package page for the exact name and version). Do not commit Autodesk DLLs to the repository.

```
dotnet add RevitAccessibility.Revit package <Revit-API-package> --version <2026-version>
```

In `RevitAccessibility.Revit.csproj` set:

```
<UseWPF>true</UseWPF>
<Platforms>x64</Platforms>
<PlatformTarget>x64</PlatformTarget>
<Nullable>enable</Nullable>
```

3.4 Core: neutral model and rules

Model (no Revit types):

```
namespace RevitAccessibility.Core.Model;

public sealed record DoorDto(string Uid, string Level, double? ClearWidthMm);
public sealed record BuildingModel(IReadOnlyList<DoorDto> Doors);

public enum Severity { Info, Warning, Error }
public enum FindingStatus { Pass, Fail, CannotEvaluate }

public sealed record Finding(
    string RuleId, string ClauseRef, Severity Severity,
    FindingStatus Status, string ElementUid, string Message);
```

Rule interface and the first rule:

```
namespace RevitAccessibility.Core.Rules;

public interface IRule
{
    string Id { get; }
    string ClauseRef { get; }
    IEnumerable<Finding> Evaluate(BuildingModel model);
}

// minWidthMm and clauseRef come from rule-register.csv, never from literals in code.
public sealed class DoorClearWidthRule(double minWidthMm, string clauseRef) : IRule
```

```
{
    public string Id => "DOOR-CLEAR-WIDTH";
    public string ClauseRef => clauseRef;

    public IEnumerable<Finding> Evaluate(BuildingModel model)
    {
        foreach (var door in model.Doors)
        {
            if (door.ClearWidthMm is null)
                yield return new Finding(Id, ClauseRef, Severity.Info,
                    FindingStatus.CannotEvaluate, door.Uid,
                    $"Door on level '{door.Level}' has no clear-width value.");
            else if (door.ClearWidthMm < minWidthMm)
                yield return new Finding(Id, ClauseRef, Severity.Error,
                    FindingStatus.Fail, door.Uid,
                    $"Clear width {door.ClearWidthMm:0} mm is below the minimum {minWidthMm:0} mm (level '{
        }
    }
}
```

Engine:

```
public sealed class RuleEngine(IEnumerable<IRule> rules)
{
    public IReadOnlyList<Finding> Run(BuildingModel model) =>
        rules.SelectMany(r => r.Evaluate(model))
             .OrderByDescending(f => f.Severity)
             .ThenBy(f => f.RuleId)
             .ToList();
}
```

Note: the engine above does not yet emit Pass findings. Decide whether Pass findings are needed for the audit trail (the UI "Show passed" toggle requires them). If yes, add a Pass branch to the rule.

3.5 Tests (write first, then implement)

Required cases for every rule:

1. Below the threshold: Fail.
2. Exactly at the threshold: no Fail.
3. One unit below the threshold: Fail.
4. Missing value: CannotEvaluate, never silently passed.
5. Empty model: no findings.
6. Mixed list: correct count per status.

```
[Fact]
public void Missing_width_is_cannot_evaluate_not_pass()
{
    var rule = new DoorClearWidthRule(800, "TEST");
    var f = Assert.Single(rule.Evaluate(new BuildingModel([new DoorDto("u1", "L0", null)])));
    Assert.Equal(FindingStatus.CannotEvaluate, f.Status);
}
```

Run: `dotnet test`. Core tests must pass on a machine without Revit installed.

3.6 Revit adapter

The adapter is the only code that touches the Revit API.

- Collect doors with `FilteredElementCollector`, category `OST_Doors`, instances only.
- Read the clear-width parameter by its verified name. Handle a missing parameter by

returning null (which becomes CannotEvaluate).

- Convert units: Revit stores lengths in feet internally. Use

`UnitUtils.ConvertFromInternalUnits(value, UnitTypeId.Millimeters)`.

- Store `UniqueId`, not `ElementId`. `UniqueId` survives saving and reopening.

Verify on a real door: an 800 mm door must read 800 mm, not 262.

3.7 Command rules

- `[Transaction(TransactionMode.ReadOnly)]`. The check must not modify the model. If a

transaction is ever opened, it is a bug.

- Catch exceptions, show a message, return `Result.Failed`.

3.8 Results window

- Modeless WPF window (so the user can keep working in Revit).
- Bind a `DataGrid` to a filtered `ICollectionView`.
- Severity and status shown as text plus icon, never colour alone.
- Detail pane for the selected finding.
- Disclaimer in the footer at all times.
- To select an element in Revit from a modeless window, use an `ExternalEvent`. Resolve

`UniqueId` to `ElementId` inside the handler's `Execute`, not in the window.

3.9 Add-in manifest

File `RevitAccessibility.addin`. Generate the `AddInId` GUID once and never change it:

```
<?xml version="1.0" encoding="utf-8"?>
<RevitAddIns>
  <AddIn Type="Command">
    <Name>Accessibility Pre-Check</Name>
    <Assembly>RevitAccessibility.Revit.dll</Assembly>
    <AddInId>GENERATE-ONCE-AND-KEEP</AddInId>
    <FullClassName>RevitAccessibility.Revit.Commands.RunCheckCommand</FullClassName>
    <VendorId>YOUR-ID</VendorId>
    <VendorDescription>Accessibility Pre-Check</VendorDescription>
  </AddIn>
</RevitAddIns>
```

The `Assembly` value should be the full path to the DLL, or a path relative to the `.addin` file as your deploy script arranges it.

3.10 Deploy

`scripts/deploy.ps1` copies the build output to:

- `%APPDATA%\Autodesk\Revit\Addins\2026\RevitAccessibility\` (DLLs)
- `%APPDATA%\Autodesk\Revit\Addins\2026\` (the `.addin` file)

Build: `dotnet build RevitAccessibility.Revit -c Debug` Deploy: `powershell -ExecutionPolicy Bypass -File scripts/deploy.ps1` Restart Revit 2026 and find the command under Add-Ins.

A 2026 add-in does not load in Revit 2024. Keep separate builds per version.

3.11 Acceptance criteria (MVP)

- The check makes no model changes (undo stack unchanged).
- Every door appears: Fail, Cannot evaluate, or Pass. None silently dropped.
- Selecting a finding selects that door in Revit.
- All Core tests pass without Revit installed.

- Validated against the reference cases in `test-model/known-results-test-cases.md`.

3.12 Repository hygiene

`.gitignore` must exclude `bin/`, `obj/`, `.vs/`, `*.user`, `*.suo`, `*.rvt`, `*.rfa`, `*.rte`, `*.ifc`, and `RevitAPI.dll` / `RevitAPIUI.dll`.

Run `git status` before the first commit and read every line.

4. Rule Verification Checklist

No threshold may be used in a real check until every step below is complete. The tool must never claim a value is verified when it is not.

4.1 Who verifies

Every rule needs two independent sign-offs:

- **Practising architect** (Italian professional registration, e.g. Ordine degli Architetti):

confirms the rule applies in real design practice and that the threshold is applied as practitioners apply it.

- **University professor** (teaching accessibility or building regulation): confirms the

article reading against the official text and the teaching explanation.

Both sign the rule register. A rule is Verified only when both have signed. Neither role replaces a formal legal opinion for a specific building.

4.2 Steps for each rule

1 **Identify the source.** Name the regulation (e.g., DM 236/1989 for accessibility of

buildings) and confirm it applies to the building type in question.

1 **Read the official text.** Use the official publication or its official source,

not a summary, forum, or AI-generated answer.

1 **Locate the article.** Record the exact article and paragraph number.

2 **Record the value.** Enter the threshold, unit, and operator exactly as written.

Note any conditions (building type, door type, exceptions).

1 **Check exceptions.** Note any exemption or alternative provision that changes the value.

2 **Record the verifier, role, and date** in the rule register (architect and professor columns).

3 **Second check.** The second verifier (the other role from 4.1) confirms steps 2 to 6.

4 **Mark status** as Verified in the rule register.

4.3 Rule register status values

Status	Meaning	Usable in real projects?
Placeholder	Value not yet checked	No
Needs review	Value entered, not yet second-checked	No
Verified	Two people have checked against the official text	Yes
Superseded	Regulation changed; kept for history	No

4.4 Re-verification

- Re-verify all rules when the regulation is amended or replaced.
- Re-verify when a project is subject to a different regional or local rule.
- Record each re-verification as a new row, not an overwrite, so history is kept.

4.5 Sign-off table (complete per rule)

RuleID	Article	Value	Unit	Entered by	Date	Checked by	Date	Status
DOOR-CLEAR-W	[TBD]	[TBD]	mm					Placeholder

4.6 Important limitations

- The register holds only rules that can be checked from model data. Many accessibility requirements (e.g., tactile paving, lighting levels, signage) cannot be checked this way.
- A pass in this tool does not mean the building complies with all requirements.

5. Roadmap and Decisions

5.1 Phases

Phase	Goal	Done when
0. Verify	Confirm rule values and parameter names	Doc 04 complete for DOOR-CLEAR-W
1. Architect workflow (Dynamo)	Working graph + Excel report	Test model gives expected results
2. Core engine (C#)	Tested rules without Revit	`dotnet test` passes without Revit
3. Revit add-in MVP	Results window and selection in Revit	Acceptance criteria in doc 03 met
4. More rules	Corridors, ramps, thresholds, WCs	Each rule verified and tested
5. Interchange	BCF export; optional IFC support	Issues open in a BCF viewer
6. Public release	GitHub sample with licence and docs	Clean repo, no client data

Phase 1 can start before phase 2. Phase 2 is needed only when the rule set grows.

5.2 Decisions made

Decision	Reason
Rules stored in a register, not hardcoded	Regulation changes without code changes; reviewable by non-developers
Core free of Revit references	Rules testable without Revit
"Cannot evaluate" is a separate status	Missing data must never look like a pass
Pre-check wording and permanent disclaimer	Avoid implying certification
Dynamo first for architects	Usable without programming skills
Revit 2026 as the first target	Matches installed software
UniqueId used for element references	Survives save and reopen
Revit only (no ArchiCAD) for now	Dynamo and Revit API only; keeps scope achievable
Free to use now; paid version planned later	Schools and students first; commercial terms to be decided before any paid release
Two verifiers per rule: practising architect and university professor	Practice and teaching checked independently
Italian regulatory base first (DM 236/1989, DPR 503/1996, UNI EN 17210), with a Jurisdiction column	Matches Italian practice and schools

5.3 Open questions

Answered:

- Revit only: yes (see 5.2).
- Free now, paid later: yes (see 5.2).
- Verifiers: practising architect and university professor (see doc 04, section 4.1).
- Building types: the rule set applies to all building types. Each rule is tagged

BuildingType = All in the register. Per-type thresholds can be added as separate rows.

- First verifier (university and professor): deferred until after the MVP.

Still open (answer before phase 1 ends):

1. Door clear width: values entered as 80 cm (entrance) and 75 cm (other doors), taken from DM 236/1989 p. 8.1.1. Still to confirm from the official text: the DPR 503/1996 article that incorporates it, and the p. 8.0.1 measurement definition. Verifiers to check.

2 What is the exact door clear-width parameter name in the project families?

3 Which licence will the GitHub sample use, given the planned paid version later?

Scope note: DPR 503/1996 governs public buildings; DM 236/1989 governs private and residential buildings. Private buildings are a later phase, and their rules go in separate rows with BuildingType = Private, using DM 236/1989 as the source.

5.5 Commercial note for the later paid version

- The Revit API is governed by Autodesk's terms. Check whether selling a tool that uses

it is permitted, and under which programme, before charging anyone.

- The licence chosen for the open GitHub sample (for example MIT) allows others to use

and sell the code. If a paid tier is planned, decide now whether the core stays open and the paid part is services, rule sets, or support, or whether the code will later become closed.

- Free use by students does not automatically permit commercial use by a firm.

5.6 Things to avoid

- Publishing client models or Autodesk DLLs.
- Adding rules before the first rule is verified and tested.
- Building a web or cloud version before the desktop tool is trusted.
- Showing a percentage score of "compliance". It implies a judgement the tool cannot make.

Rule Register (rules/rule-register.csv)

Edit values in the CSV file, not in the code. Placeholder rows must not be used on real projects.

Rule: DOOR-CLEAR-W-ENTRY (Needs review)

- **Jurisdiction:** IT-National
- **BuildingType:** Public
- **Clause:** DM 236/1989 p. 8.1.1 (via DPR 503/1996; article reference to confirm)
- **Category:** Doors
- **Parameter:** Clear Width
- **Operator:** >=
- **Threshold:** 80
- **Unit:** cm
- **Severity:** Error
- **Status:** Needs review
- **VerifiedBy_Architect:** (blank)
- **VerifiedBy_Professor:** (blank)
- **Date:** (blank)
- **Notes:** Entrance/access door of building or unit. Values match several secondary sources; official text not yet read. Measurement: luce netta per DM 236 p. 8.0.1 (to confirm). Needs door-type classification in the model.

Rule: DOOR-CLEAR-W-OTHER (Needs review)

- **Jurisdiction:** IT-National
- **BuildingType:** Public
- **Clause:** DM 236/1989 p. 8.1.1 (via DPR 503/1996; article reference to confirm)
- **Category:** Doors
- **Parameter:** Clear Width
- **Operator:** >=
- **Threshold:** 75
- **Unit:** cm
- **Severity:** Error
- **Status:** Needs review
- **VerifiedBy_Architect:** (blank)
- **VerifiedBy_Professor:** (blank)
- **Date:** (blank)
- **Notes:** Other doors. Values match several secondary sources; official text not yet read. Needs door-type classification in the model.

Rule: CORR-W (Placeholder)

- **Jurisdiction:** IT-National
- **BuildingType:** Public

- **Clause:** [TBD - DPR 503/1996 article]
- **Category:** Corridors
- **Parameter:** Width
- **Operator:** >=
- **Threshold:** [TBD]
- **Unit:** mm
- **Severity:** Error
- **Status:** Placeholder
- **VerifiedBy_Architect:** (blank)
- **VerifiedBy_Professor:** (blank)
- **Date:** (blank)
- **Notes:** Threshold not verified. Do not use in real projects. Scope to be confirmed by verifiers (public buildings first).

Rule: RAMP-SLOPE (Placeholder)

- **Jurisdiction:** IT-National
- **BuildingType:** Public
- **Clause:** [TBD - DPR 503/1996 article]
- **Category:** Ramps
- **Parameter:** Slope
- **Operator:** <=
- **Threshold:** [TBD]
- **Unit:** %
- **Severity:** Error
- **Status:** Placeholder
- **VerifiedBy_Architect:** (blank)
- **VerifiedBy_Professor:** (blank)
- **Date:** (blank)
- **Notes:** Threshold not verified. Do not use in real projects. Scope to be confirmed by verifiers (public buildings first).

Rule: THRESH-H (Placeholder)

- **Jurisdiction:** IT-National
- **BuildingType:** Public
- **Clause:** [TBD - DPR 503/1996 article]
- **Category:** Doors
- **Parameter:** Threshold Height
- **Operator:** <=
- **Threshold:** [TBD]
- **Unit:** mm

- **Severity:** Warning
- **Status:** Placeholder
- **VerifiedBy_Architect:** (blank)
- **VerifiedBy_Professor:** (blank)
- **Date:** (blank)
- **Notes:** Threshold not verified. Do not use in real projects. Scope to be confirmed by verifiers (public buildings first).

Known-Results Test Cases

Use a test model you created yourself (never a client model). Each door below must produce exactly the stated result. Values are for testing the checker only; they are not regulation thresholds.

Test thresholds used here: **minimum 800 mm** (demo value only, not verified).

Test door	Level	Clear width (mm)	Expected status	Expected message type
T-01	L0	700	Fail	Below minimum
T-02	L0	800	Pass	Meets minimum (boundary)
T-03	L0	799	Fail	Below minimum (one unit under)
T-04	L1	900	Pass	Meets minimum
T-05	L1	(parameter absent)	Cannot evaluate	Missing data
T-06	L1	(parameter present, empty)	Cannot evaluate	Missing data
T-07	L2	1	Fail	Very small value (checks unit handling)

Unit check

Create one door 800 mm wide in the model. The check must read 800, not 262 (feet misread) and not 0.8 (metres misread). If the value is wrong, stop and fix the unit conversion before any other test.

Pass criteria

- Every test door appears in the output.
- Each status matches the expected column.
- No door is missing from the output.
- The check leaves the model unchanged.

Record of runs

Date	Revit version	Checker version	Result	Tester